Chapitre 0 — Avant tout | Partie C — Déplacer une lettre | leçon 0.29.4 sur 153 | difficulté 0 / 10

0.29.4. La fonction un_pas() — un seul pas, sans attendre

#include <un_pas> : un_pas(x, y, tuile, sensX, sensY) fait faire UN pas à la tuile, tout de suite, sans bloquer le jeu.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `un_pas(x, y, tuile, sensX, sensY)` fait faire **un seul pas**, tout de suite, sans attendre : 1 vers la droite ou le bas, -1 vers la gauche ou le haut, 0 immobile. Elle ne bloque pas : on l'appelle dans la boucle, et d'autres choses bougent en même temps.

La ligne à écrire : `#include <un_pas>`. Ici aussi `<chaque>`, pour rythmer les pas.

Ses arguments : la place, la tuile, le sens en x et en y. Seule sur sa ligne, elle range la nouvelle place dans x et y.

Ce qu'elle coûte : environ **180 octets**, plus la police entière.

Essaie : `un_pas(x, y, ALPHABET[0], 1, 1);` : en diagonale.

> **On retrouve un_pas dans 15 leçons**
>
> 0.30, 0.30.1, 0.30.2, 0.31, 0.31.1, 0.31.2, 0.76.2, 0.76.3, 0.76.4, 0.76.5, 0.76.6, 0.79, 0.79.1, 0.84, 0.85

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **opérateurs logiques** — && « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <un_pas> : un seul pas, sans attendre ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <un_pas> : un seul pas, sans attendre ---- »). La console l'ignore.
`// Toutes les 250 ms, le A fait un pas vers la droite, tant qu'il n'est pas au bord.`	Un commentaire : une phrase pour celui qui lit (« Toutes les 250 ms, le A fait un pas vers la droite, tant qu'il n'est pas au bord. »). La console l'ignore.

`#include <un_pas>     // un_pas() : un pas, tout de suite`	Demande à la console « un_pas » (ce que c'est : fait faire un seul pas à une tuile). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « un_pas() : un pas, tout de suite ».)
`#include <chaque>`	Demande à la console « chaque » (ce que c'est : répond 1 toutes les n millisecondes). Sans cette ligne, le compilateur refuserait de l'employer.
`#include <ALPHABET>`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer.
`uint8_t x = 0;`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0.
`uint8_t y = 8;`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 8.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`if (chaque(250) && x < 19) {`	Une décision : si chaque(250) et x est plus petit que 19, on fait ce qui est entre les accolades. Sinon, on passe.
`un_pas(x, y, ALPHABET[0], 1, 0);   // un pas à droite ; x grandit de 1`	Appel de la fonction un_pas de la console, avec les arguments x, y, ALPHABET[0], 1, 0. Ce qu'elle fait : UN pas, tout de suite, sans attendre : 1 vers la droite ou le bas, -1 vers la gauche ou le haut, 0 immobile. Ne bloque pas : à appeler dans la boucle, pour faire bouger plusieurs choses à la fois. un_pas(x, y, ALPHABET[0], 1, 0); range la nouvelle place dans x et y. (Le commentaire le dit : « un pas à droite ; x grandit de 1 ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1182 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage           63 o  ← la console, à l'allumage  — la mise en route de la console
  chaque_minuteur()  124 o  ← main()  — une fonction fournie par le langage
  un_pas()           117 o  ← main()  — une fonction fournie par le langage
  main()              56 o  ← démarrage  — ta fonction
  VBlank               8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage       28 o  ← image()  — attend l'image suivante
  AttendreVBlank      16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces       12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase         10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases        12 o  ← effacer()  — efface des cases
```

```
  CopierTuiles        18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte        14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles             704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  effacer()         × 1    15 o sur place, dans un_pas()  + AdresseCase (10 o), EffacerCases (12 o)
  poser()           × 1    13 o sur place, dans un_pas()  + AdresseCase (10 o)
  image()           × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  images()          × 1     2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
  un_pas()          × 1    22 o sur place, dans main()
  chaque_minuteur() × 1    10 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <un_pas> : un seul pas, sans attendre ----
// Toutes les 250 ms, le A fait un pas vers la droite, tant qu'il n'est pas au bord.

#include <un_pas>     // un_pas() : un pas, tout de suite
#include <chaque>
#include <ALPHABET>
```

2. UNE VARIABLE — X

```
uint8_t x = 0;
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

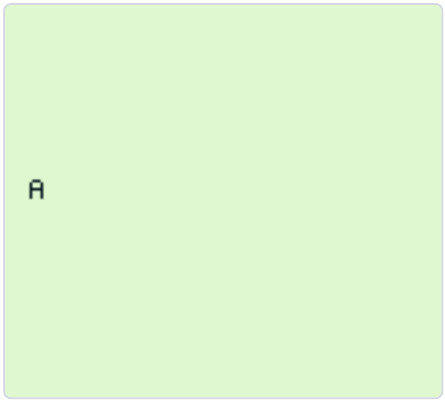
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();
    if (chaque(250) && x < 19) {
      un_pas(x, y, ALPHABET[0], 1, 0);   // un pas à droite ; x grandit de 1
    }
  }
}
```

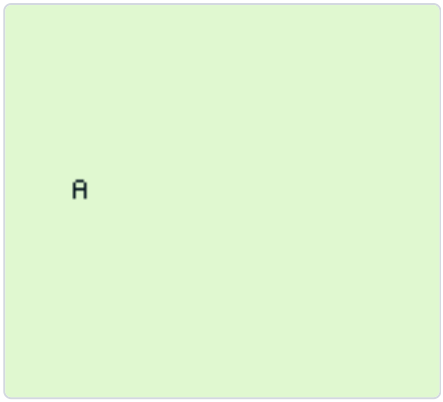
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

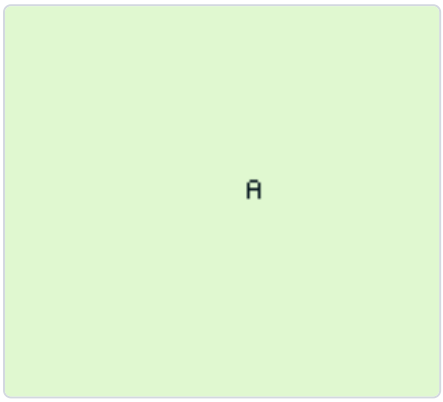
la toute première image — la console vient de démarrer — image 1



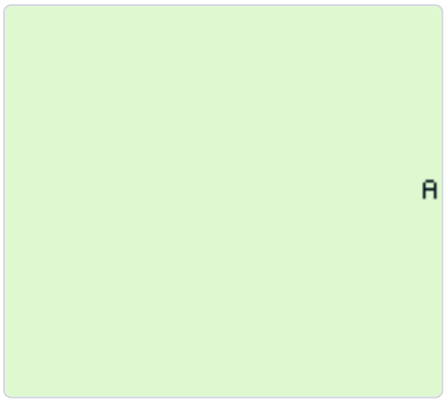
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Un A qui avance d'une case tous les quarts de seconde, jusqu'au bord droit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A avance case par case (9 places)

✓ vers la droite, sur la ligne 8